Keeping a System Healthy Under Pressure

Load Shedding, Throttling, Rate Limiting, Retries with Backoff, and Timeouts

A plain-language explanation of the project, its components, its workflow and its architecture

> **The project statement**
> "Sustained SLOs through 5x traffic surges and network partitions with zero cascading failures, as measured by brownout and fault-injection tests, by implementing load shedding, throttling, rate limiting, retries with backoff, and timeouts."

Section	What you will find
1. Summary in simple terms	The whole project explained in a few minutes
2. Key words explained	A small dictionary of every technical term used
3. Components	Every building block, explained end to end
4. Architecture diagram	The picture of the system and every box, arrow and number in it
5. Workflow	The life of a single request, then full surge and partition scenarios
6. Testing and proof	How brownout and fault-injection tests show the goals were met
7. Quick reference	A one-page cheat sheet

1. Summary in Simple Terms

Imagine a popular restaurant. On a normal evening, the kitchen handles everyone easily. Then one night five times as many people show up at once, and on top of that the phone line to the main food supplier keeps cutting out. A badly run restaurant panics: orders pile up, the kitchen jams, waiters keep re-shouting the same orders, and soon **nobody** gets food. A well-run restaurant stays calm: a host at the door controls how many people come in, the kitchen only takes as many orders as it can actually cook, simple dishes are served first when things get tight, and staff stop calling the supplier when the line is dead and use what is in the fridge instead.

This project did exactly that for a software system. The system is a set of online services (the "kitchen") that receive requests from users and partner companies (the "customers"). The goal was to keep the service working well even in two bad situations:

- **A 5x traffic surge**: suddenly five times the normal number of requests arrive (for example, a big sale, a viral moment, or a misbehaving client).
- **A network partition**: parts of the system temporarily cannot talk to each other, or the connection becomes very slow and lossy.

To do this, five protective techniques were built in:

Technique	Plain meaning	Restaurant version
Rate limiting	Each caller gets a fixed number of requests per second	Each table may only order a certain number of dishes per minute
Throttling	Only a fixed amount of work runs at once; extra work waits briefly or is turned away	The kitchen only cooks as many dishes as it has stoves
Load shedding	When the system is overloaded, drop the least important work first	When swamped, stop serving desserts so mains still go out
Timeouts	Never wait forever for an answer	If the supplier does not pick up in 30 seconds, hang up
Retries with backoff	Try again after a failure, but wait longer each time and add randomness	Call back in 1 minute, then 2, then 4, not every second

The outcome had three parts:

- **SLOs sustained**: the service kept meeting its promised targets (its Service Level Objectives, such as "99.9% of requests succeed" and "most requests answer in under 300 milliseconds") during the stress.
- **Zero cascading failures**: a problem in one part never knocked over the other parts in a chain reaction.
- **Proven by tests**: this was not just claimed; it was measured by deliberately creating overload (brownout tests) and deliberately breaking the network (fault-injection tests), then checking the numbers.

> **The single most important idea**
>
> It is better to say a quick, polite "not right now" to some requests than to say a slow "yes" to all of them and then crash. A system that rejects 20% of extra traffic quickly stays healthy for the other 80%, and for all the normal traffic. A system that tries to accept everything falls over for everyone.

2. Key Words Explained

These are the words used throughout the document. Each is explained in everyday language.

Term	Simple meaning
Request	One message asking the system to do something, like "show my orders" or "pay for this cart".
Service	A program that does one job, like handling orders or payments. Big systems are made of many services that call each other.
Dependency	Something a service needs to call to finish its job: a database, a cache, another service, or an outside company's API.
SLO (Service Level Objective)	A target the team promises to hit, such as "99.9% of requests succeed" or "95% of requests finish within 300 ms".
SLI (Service Level Indicator)	The actual measured number compared against the SLO, such as "right now 99.95% are succeeding".
Error budget	The small amount of failure the SLO allows. 99.9% success means 0.1% of requests may fail. That 0.1% is the budget.
Latency	How long a request takes from start to finish. Often reported as p50 (typical), p95 and p99 (the slow tail).
Throughput	How many requests are handled per second.
Capacity	The most work the system can do well before it starts slowing down.
Traffic surge (5x)	A sudden jump to five times the normal request rate.
Network partition	When parts of the system cannot reach each other, fully or partly (dropped or very slow messages).
Cascading failure	A chain reaction: one part slows or fails, its callers pile up waiting, they fail too, and so on until everything is down.
Retry storm	When many clients retry failed requests at the same moment, multiplying traffic and making the outage worse.
Brownout test	A test that pushes the system into heavy overload on purpose to see if it degrades gracefully instead of blacking out.

Term	Simple meaning
Fault-injection test	A test that breaks things on purpose (cut network links, add delay, drop packets) to check the system survives.
HTTP 429	The standard reply meaning "Too Many Requests, slow down".
HTTP 503	The standard reply meaning "Service Unavailable right now, try later".
Retry-After	A hint in the reply telling the caller how many seconds to wait before trying again.
Idempotent	Safe to repeat. Reading data is idempotent. Charging a card twice is not, unless protected by an idempotency key.
Jitter	A small random amount added to wait times so that many clients do not all retry at the exact same moment.
Graceful degradation	Giving a simpler or partial answer instead of a total failure, such as showing cached data.

3. Components Explained End to End

This section explains every building block. Each component follows the same pattern so it is easy to compare: a one-line meaning, an everyday analogy, why it is needed, how it works step by step, its settings, what the caller sees, where it lives, and the mistakes to avoid.

#	Component	Main job	Protects against
3.1	Rate limiter	Caps requests per caller per second	One noisy caller hogging everything
3.2	Throttler (concurrency limiter)	Caps how much work runs at once	Too much work in flight, memory and thread exhaustion
3.3	Load shedder	Drops low-priority work when overloaded	Total collapse during surges
3.4	Timeouts and deadlines	Puts a time limit on every wait	Stuck requests piling up
3.5	Retries with backoff and jitter	Recovers from brief glitches safely	Temporary blips becoming errors
3.6	Retry budget	Limits total extra traffic caused by retries	Retry storms
3.7	Circuit breaker	Stops calling a dependency that is clearly broken	Wasting time on dead calls
3.8	Fallbacks	Gives a useful partial answer	Users seeing hard errors
3.9	Limits store and configuration	Shares counters and settings	Inconsistent limits across servers
3.10	Observability (metrics, dashboards, alerts)	Shows what is happening	Flying blind
3.11	Test harness (brownout and fault injection)	Proves it all works	Untested assumptions

3.1 Rate Limiter

> **In one line**

> A rule that says each caller (a user, an app, an API key) may only send a set number of requests per second, and anything above that is politely turned away.

Everyday analogy

A ticket machine at a busy deli. Each customer gets tickets at a steady pace. If you have a ticket, you can order. If you run out, you wait until the machine gives you another. One person cannot grab 100 tickets and block everyone.

Why we need it

- Stops one heavy or buggy caller from using up the capacity that everyone shares.
- Makes the load the system receives predictable, which makes the other protections easier to tune.
- Gives fair access: each customer or partner gets their agreed share.

How it works, step by step

1. A request arrives at the front door (the API gateway).
2. The gateway works out **who** is calling, using the API key, user ID or IP address.
3. It looks up that caller's **bucket** in the shared limits store. Think of the bucket as a jar of tokens.
4. Tokens drip into the jar at a fixed rate (the refill rate, for example 100 per second) up to a maximum (the burst size, for example 200).
5. If the jar has at least one token, one token is taken out and the request is allowed through.
6. If the jar is empty, the request is rejected straight away with **HTTP 429 Too Many Requests** and a **Retry-After** hint.
7. The decision (allowed or rejected) is counted in the metrics so the team can see who is being limited and how often.

Key settings (the knobs)

Setting	What it means	Typical starting value
Refill rate	How many tokens are added per second	Agreed per-customer quota, e.g. 100/s
Burst size	How many tokens the jar can hold, allowing short bursts	About 2x the refill rate
Key	What identifies a caller	API key, else user ID, else IP

Setting	What it means	Typical starting value
Scope	Per server or shared across all servers	Shared (via the limits store)

What the caller sees

Normal callers never notice it. A caller who goes over their limit gets a fast 429 reply with a Retry-After value telling them exactly when to come back. The reply is fast and cheap, so even a flood of rejected requests does not hurt the system.

Where it lives in the system

At the very front of the system, in the API gateway (Layer 1 in the architecture diagram, number 2). It is checked first because it is the cheapest check and protects everything behind it.

Common mistakes to avoid

- Keeping counters only on one server, so a caller can get N times the limit by spreading requests across N servers.
- Forgetting the Retry-After hint, which makes clients guess and retry too soon.
- Setting limits from guesses rather than from measured capacity.

3.2 Throttler (Concurrency Limiter)

> **In one line**
>
> A cap on how many requests are being worked on at the same moment, with a small, short waiting line for a few extras and a quick "no" for the rest.

Everyday analogy

A kitchen with 10 stoves. Only 10 dishes can cook at once. A couple of orders can wait on the counter for a moment, but if the counter fills up, new orders are declined instead of piling up forever.

Why we need it

- Rate limiting controls **arrivals** per caller, but the system as a whole can still get too many requests at once. Throttling controls the **total work in progress**.
- Every in-flight request uses memory, threads and connections. Too many at once makes everything slow, which is the start of a cascading failure.
- Keeping work in progress near the sweet spot keeps latency low and throughput high.

How it works, step by step

1. After passing the rate limiter, the request asks for a **work slot**.
2. If a slot is free (for example, fewer than 200 requests running), the request takes it and moves on.
3. If all slots are busy, the request joins a **small, bounded queue** (for example, at most 50 waiting).
4. It waits only a short time (for example, up to 50 ms). Waiting longer would just waste the user's time.
5. If a slot frees up in time, it proceeds. If not, or if the queue is already full, it is rejected with **HTTP 503** and a Retry-After hint.
6. When the request finishes, it gives its slot back so the next one can start.
7. In the adaptive version, the slot count adjusts itself: if latency rises, the limit shrinks a little; if latency is healthy, it grows a little (similar to how TCP adjusts its sending speed).

Key settings (the knobs)

Setting	What it means	Typical starting value
Max concurrent	Slots available at once	Measured from load tests (the point where latency starts rising)
Queue size	How many may wait	Small, e.g. 25% of max concurrent
Max queue wait	Longest time to wait for a slot	A small fraction of the timeout, e.g. 50 ms
Adaptive	Whether the limit tunes itself	On, based on latency

What the caller sees

Under normal load, nothing. Under heavy load, a few requests may take a few milliseconds longer. When truly full, a quick 503 instead of a long hang.

Where it lives in the system

In the API gateway (number 3) and also inside each service, so each service protects itself even from internal callers.

Common mistakes to avoid

- Using a huge or unlimited queue. A long queue only hides overload and makes every request slow.
- Letting requests wait in the queue longer than the user is willing to wait.

3.3 Load Shedder

> **In one line**
> When the system detects it is overloaded, it deliberately drops some requests, starting with the least important ones, so the important ones still succeed.

Everyday analogy

A hospital emergency room on a very busy night uses triage: life-threatening cases first, minor cuts wait or are sent to a clinic. Nobody is ignored on purpose, but the most important work is protected.

Why we need it

- Rate limiting and throttling use fixed numbers. Load shedding reacts to the **actual health** of the system right now (CPU, queue length, latency).
- During a 5x surge, even legitimate traffic exceeds capacity. Something has to give, and it is better to choose what than to let everything fail randomly.
- Shedding early and cheaply keeps the server out of the "death spiral" where it spends all its time on requests that will time out anyway.

How it works, step by step

1. Every request is tagged with a **priority**. For example: Critical (checkout, payments), High (logged-in browsing), Normal (search, recommendations), Low (background jobs, analytics, prefetching).
2. The load shedder constantly reads **health signals**: CPU usage, number of in-flight requests, queue wait time, and recent latency compared with the target.
3. It turns these into a simple **pressure level**, for example Green, Yellow, Orange, Red.
4. Green: accept everything.
5. Yellow: drop Low priority work.
6. Orange: also drop Normal priority work, or answer it from cache instead.
7. Red: only Critical work is accepted.
8. Dropped requests get a fast **503** (or a cached or simplified answer). Dropping is decided at the front, before any expensive work is done.
9. It also drops requests whose **deadline has already passed or is about to pass**, because finishing them would be wasted work.
10. When the signals return to normal, the level steps back down gradually (not instantly) so it does not flip back and forth.

Key settings (the knobs)

Setting	What it means	Typical starting value
Priority classes	Which kinds of requests matter most	Critical / High / Normal / Low
Thresholds	Signal values that move between levels	e.g. CPU 70% / 80% / 90%, or queue wait 10 / 30 / 50 ms
Cool-down	How long to stay at a level before stepping down	e.g. 30 seconds

What the caller sees

Important users (like someone checking out) keep getting fast, normal responses. Less important features may briefly show a cached version, a simpler page, or a "try again shortly" message.

Where it lives in the system

In the API gateway (number 4) and inside each service. The front copy protects the whole system; the inner copies protect each service individually.

Common mistakes to avoid

- Shedding too late, after the server is already struggling. Shedding must be cheap and early.
- Treating all traffic as equally important, which makes it impossible to protect what matters.

3.4 Timeouts and Deadlines

> **In one line**
>
> Every wait has a time limit, and the remaining time is passed along to each step so no part of the system keeps working on a request that the user has already given up on.

Everyday analogy

You give a delivery driver until 6 pm. The driver tells the warehouse "I need it by 5:30". The warehouse tells the packer "I need it by 5:15". If the packer cannot make it, they stop early instead of packing something nobody will receive.

Why we need it

- Without a timeout, one slow dependency makes callers wait forever. Their threads and connections fill up, and they become slow too. This is the most common cause of cascading failures.
- During a network partition, messages can vanish. A timeout is the only way to notice and move on.
- Deadlines stop wasted work: if the user has already left, there is no point finishing the job.

How it works, step by step

1. When a request enters the gateway, it gets an overall **deadline**, for example 2 seconds from now.
2. As the request moves from service to service, the **remaining time** travels with it (for example, in a request header).
3. Before calling a dependency, the service sets a **per-call timeout** that is the smaller of: the normal timeout for that dependency, or the remaining time minus a small safety margin.
4. There are separate timeouts for **connecting** (short, for example 100 ms) and for **waiting for the answer** (longer, based on that dependency's normal p99 latency plus a margin).
5. If the answer does not arrive in time, the call is cancelled, its resources are freed, and the error is handed to the retry logic.
6. If the remaining time is already too small to do useful work, the service skips the call and returns a fallback or error immediately.

Key settings (the knobs)

Setting	What it means	Typical starting value
Overall deadline	Total time a request may take	e.g. 2 s for user-facing calls
Connect timeout	Time to open a connection	e.g. 100 ms
Per-call timeout	Time to wait for each dependency answer	Dependency p99 latency + margin
Safety margin	Time kept back for replying	e.g. 50 ms

What the caller sees

A response always arrives within a predictable time: either the real answer, a fallback, or a clear error. Never an endless spinner.

Where it lives in the system

In the resilience library that wraps every outgoing call (number 7), and at the gateway where the deadline is first set.

Common mistakes to avoid

- Inner timeouts longer than outer ones, so the caller gives up while the inner work continues uselessly.
- Using one timeout value for everything instead of tuning it per dependency.
- No timeout at all on database or HTTP clients (some libraries default to waiting forever).

3.5 Retries with Exponential Backoff and Jitter

> **In one line**
>
> When a call fails for a temporary reason, try again, but wait longer between each try and add a bit of randomness, and only when it is safe.

Everyday analogy

If a friend's phone is busy, you try again in a minute, then two minutes, then four, not every second. And if a whole crowd is calling the same person, each waits a slightly different time so they do not all ring at once.

Why we need it

- Many failures are brief: a dropped packet, a server restarting, a short network hiccup. A retry often succeeds and the user never notices.
- But careless retries are dangerous. If every client retries instantly, a struggling service gets two, three or four times the traffic just when it is weakest. This is a **retry storm**.
- Backoff and jitter spread retries out over time so the service gets breathing room to recover.

How it works, step by step

1. A call fails (timeout, connection reset, 503, or 429).
2. **Is it worth retrying?** Only temporary errors are retried. Errors like "invalid input" (400) or "not allowed" (403) are never retried because they will fail the same way again.
3. **Is it safe to retry?** Only if the operation is idempotent (safe to repeat), or it carries an **idempotency key** so the server can recognise and ignore a duplicate (for example, so a payment is never charged twice).
4. **Is there budget left?** The retry budget (section 3.6) must allow it.
5. **Is there time left?** The remaining deadline must be long enough for another attempt.

6. If all four answers are yes, wait using **exponential backoff**: base delay x 2 to the power of the attempt number, for example 100 ms, 200 ms, 400 ms, capped at a maximum such as 2 seconds.
7. Add **jitter**: pick a random wait between 0 and that number ("full jitter"), so different clients spread out.
8. If the server sent a **Retry-After** value, respect it instead of the computed wait.
9. Try again. Stop after a small maximum number of attempts (for example, 3 in total).
10. If all attempts fail, hand the error to the fallback logic.

Key settings (the knobs)

Setting	What it means	Typical starting value
Max attempts	Total tries including the first	3
Base delay	First wait	100 ms
Max delay	Longest single wait	2 s
Jitter	Randomness method	Full jitter: random(0, computed delay)
Retryable errors	Which failures to retry	Timeouts, connection errors, 503, 429

What the caller sees

Brief glitches are invisible to users because the retry quietly succeeds. During a real outage, the user gets a timely fallback instead of being stuck through endless retries.

Where it lives in the system

In the resilience library (number 7) in each service, and also in the client apps (web and mobile) which follow the Retry-After hints.

Common mistakes to avoid

- Retrying at every layer. If 3 layers each retry 3 times, one user request can become 27 calls at the bottom. Retry at one layer only, usually the one closest to the failing dependency.
- Retrying non-idempotent operations without an idempotency key.
- Fixed delays with no jitter, which make clients retry in synchronized waves.

3.6 Retry Budget

> **In one line**
>
> A cap on how much extra traffic retries are allowed to add, for example retries may be at most 10% of normal requests.

Everyday analogy

A shop gives each cashier a small number of "redo" tokens per hour. Once they are used up, any further problems are handled differently rather than redoing transactions endlessly.

Why we need it

- Per-request retry limits (3 attempts) are not enough. If everything is failing, 3 attempts still means 3x the traffic.
- A budget makes sure that during a big outage retries fade away automatically, and during small glitches they still help.

How it works, step by step

1. Each service keeps a rolling count of normal requests and of retries over the last few seconds.
2. Before any retry, it checks: is retries / requests still under the budget (for example, 10%)?
3. If yes, the retry may go ahead (subject to the other checks in 3.5).
4. If no, the retry is skipped and the error goes straight to the fallback.
5. As the outage ends and successes return, the ratio falls and retries are allowed again.

Key settings (the knobs)

Setting	What it means	Typical starting value
Budget ratio	Max retries as a share of requests	10%
Window	Time period counted	10 seconds
Minimum allowance	Small floor so low-traffic services can still retry	e.g. 10 retries per second

What the caller sees

Nothing directly. It quietly prevents the retry storms that would otherwise turn a small issue into a big one.

Where it lives in the system

In the resilience library (number 7) next to the retry logic.

3.7 Circuit Breaker (supporting component)

> **In one line**
>
> If a dependency keeps failing, stop calling it for a short while and fail fast, then test carefully whether it has recovered.

Everyday analogy

The circuit breaker in your house: when there is a fault, it trips and cuts the power so the wiring does not burn. After a while, you flip it back on and see if things are okay.

Why we need it

- Even with timeouts, every call to a dead dependency still wastes the full timeout. A breaker turns a slow failure into an instant one.
- It gives the failing dependency room to recover instead of being hammered.
- It makes behaviour during a network partition fast and predictable.

How it works, step by step

1. **Closed** (normal): calls go through. The breaker counts failures over a rolling window.
2. If the failure rate passes a threshold (for example, 50% of at least 20 calls), the breaker **opens**.
3. **Open**: calls are not sent at all. They fail immediately and go to the fallback. This lasts for a cool-down period (for example, 10 seconds).
4. **Half-open**: after the cool-down, a few trial calls are let through.
5. If the trials succeed, the breaker closes again and normal traffic resumes. If they fail, it opens again for another cool-down.

Key settings (the knobs)

Setting	What it means	Typical starting value
Failure threshold	Error rate that trips the breaker	50%
Minimum calls	Calls needed before judging	20
Open duration	Cool-down before testing	10 s
Trial calls	Calls allowed in half-open	5

What the caller sees

During a dependency outage, users get an instant fallback (for example, cached data) rather than waiting for a timeout on every request.

Where it lives in the system

In the resilience library (number 7), one breaker per dependency.

3.8 Fallbacks (Graceful Degradation)

> **In one line**
>
> A plan B answer used when the real answer is not available in time.

Everyday analogy

If the coffee machine breaks, a café offers filter coffee instead of closing the shop.

Why we need it

- Turns many errors into acceptable experiences.
- Lets the system keep meeting its SLOs (which count successful answers) even when a non-essential dependency is down.

How it works, step by step

1. Each dependency call declares what to do if it fails.
2. Options include: return slightly old data from the **cache**; return a **default** (for example, generic recommendations); **hide** an optional part of the page; or **queue** the work to finish later (for example, sending a confirmation email).
3. For work that truly cannot be faked (like taking a payment), return a clear, honest error quickly so the user can retry later.
4. The fallback is recorded in metrics so the team knows how often it is used.

What the caller sees

A page that still works, perhaps with slightly older information or one missing section, instead of an error page.

Where it lives in the system

In each service, wired to the resilience library (numbers 7 and 8), using the cache and message queue in Layer 3.

3.9 Limits Store and Configuration

> **In one line**
>
> A shared, fast place (such as Redis) where rate limit counters and protection settings live, so every server enforces the same rules.

Everyday analogy

A shared whiteboard in the restaurant that every host checks, so all doors follow the same count of how many guests are inside.

Why we need it

- Systems run on many servers. Without a shared counter, each server would allow the full limit and the true limit would be multiplied.
- Settings (limits, thresholds, timeouts) can be changed quickly without redeploying code, which is vital during an incident.

How it works, step by step

1. Gateways read and update token buckets in the store with small, atomic operations (so two servers never take the same token).
2. Each gateway also keeps a tiny local cache of recent decisions to avoid a store call on every single request.
3. If the store itself is unreachable, gateways **fail safe**: they fall back to local per-server limits rather than either blocking everyone or allowing unlimited traffic.
4. Configuration changes are versioned and rolled out gradually.

What the caller sees

Consistent, fair limits no matter which server they reach.

Where it lives in the system

In the Control and Watch column (number 9), connected to the gateway.

3.10 Observability: Metrics, SLO Dashboard and Alerts

> **In one line**
>
> The instruments that show, in real time, whether the system is meeting its promises and which protections are active.

Everyday analogy

The dashboard in a car: speed, fuel and warning lights. You cannot drive safely without it.

Why we need it

- You cannot claim SLOs were sustained without measuring them.
- Shows whether the protections are working (for example, how many requests are shed) and whether they are too strict or too loose.
- Alerts wake people up only when user experience is actually at risk.

How it works, step by step

1. Every component records numbers: request count, success and error counts, latency (p50, p95, p99), rejections by reason (rate limit, throttle, shed), retries, timeouts, breaker state, fallback usage.
2. These are collected by a metrics system (for example, Prometheus).
3. The SLO dashboard (for example, Grafana) compares the measured SLIs to the SLO targets and shows the **error budget**: how much failure is still allowed this month.
4. It also shows **burn rate**: how fast the budget is being used. A burn rate of 1 means on track; 14 means the month's budget would be gone in about 2 days.
5. Alerts fire on fast burn (urgent page) and slow burn (ticket), not on every small spike.

What the caller sees

Nothing directly; this is for the engineering team.

Where it lives in the system

In the Control and Watch column (numbers 10), receiving data from every layer.

3.11 Test Harness: Brownout and Fault-Injection Tests

> **In one line**
>
> Tools that deliberately overload and break the system in a controlled way, then check that the SLOs held and nothing cascaded.

Everyday analogy

A fire drill. You do not wait for a real fire to find out whether the exits work.

Why we need it

- Protections that have never been tested often fail in real incidents (wrong thresholds, missing timeouts, retries in the wrong place).

- Tests produce the hard numbers behind the project claim: "as measured by brownout and fault-injection tests".

How it works, step by step

1. **Load generator** (for example, k6, Locust, or Gatling) replays realistic traffic with the real mix of request types and priorities.
2. **Brownout test**: start at normal load, then ramp up to 2x, 3x and 5x, hold, then ramp back down. Optionally reduce capacity (remove servers) to create an even harsher squeeze.
3. **Fault injector** (for example, Toxiproxy, Chaos Mesh, Gremlin, or Linux tc/netem) adds faults between services and dependencies: extra latency, packet loss, dropped connections, and full partitions where one side cannot reach the other.
4. **SLO verifier** reads the metrics during and after each test and compares them with pass/fail criteria (see section 6).
5. Results are saved and the tests are re-run regularly (for example, in a staging environment before each major release) so protections do not quietly break over time.

What the caller sees

Nothing; tests run in staging or in carefully controlled production experiments.

Where it lives in the system

Layer 4 at the bottom of the diagram (number 11).

3.12 How the Components Work Together

No single technique is enough. They form layers of defense, each catching what the one before missed:

Layer of defense	Question it answers	If the answer is bad
Rate limiter	Is this caller sending more than their fair share?	429, come back later
Throttler	Is the system already doing as much as it safely can?	Short wait, then 503
Load shedder	Is the system under stress, and is this request important enough?	503 or cached answer, low priority first
Timeout / deadline	Is this wait taking too long?	Cancel and free resources
Retry with backoff	Was it a brief glitch worth trying again?	Wait (growing, random), then retry
Retry budget	Are retries starting to add too much traffic?	Stop retrying, use fallback
Circuit breaker	Is this dependency clearly down?	Fail fast, use fallback
Fallback	What is the best answer we can still give?	Cached, default, or partial answer

> **Why this prevents cascading failures**
>
> A cascading failure needs two things: work piling up (queues and waiting threads), and that pile-up spreading to callers. Throttling and load shedding stop the pile-up from forming. Timeouts and circuit breakers stop a slow dependency from holding callers hostage. Retry budgets and jitter stop the recovery attempts from becoming a new overload. Together, every failure stays contained in the place where it started.

4. Architecture Diagram

The diagram below shows the whole system. Read it from top to bottom: requests start with the callers at the top, pass through the front door, reach the services, and finally touch the data and dependencies. The right-hand column watches everything, and the test harness at the bottom puts it all under stress.

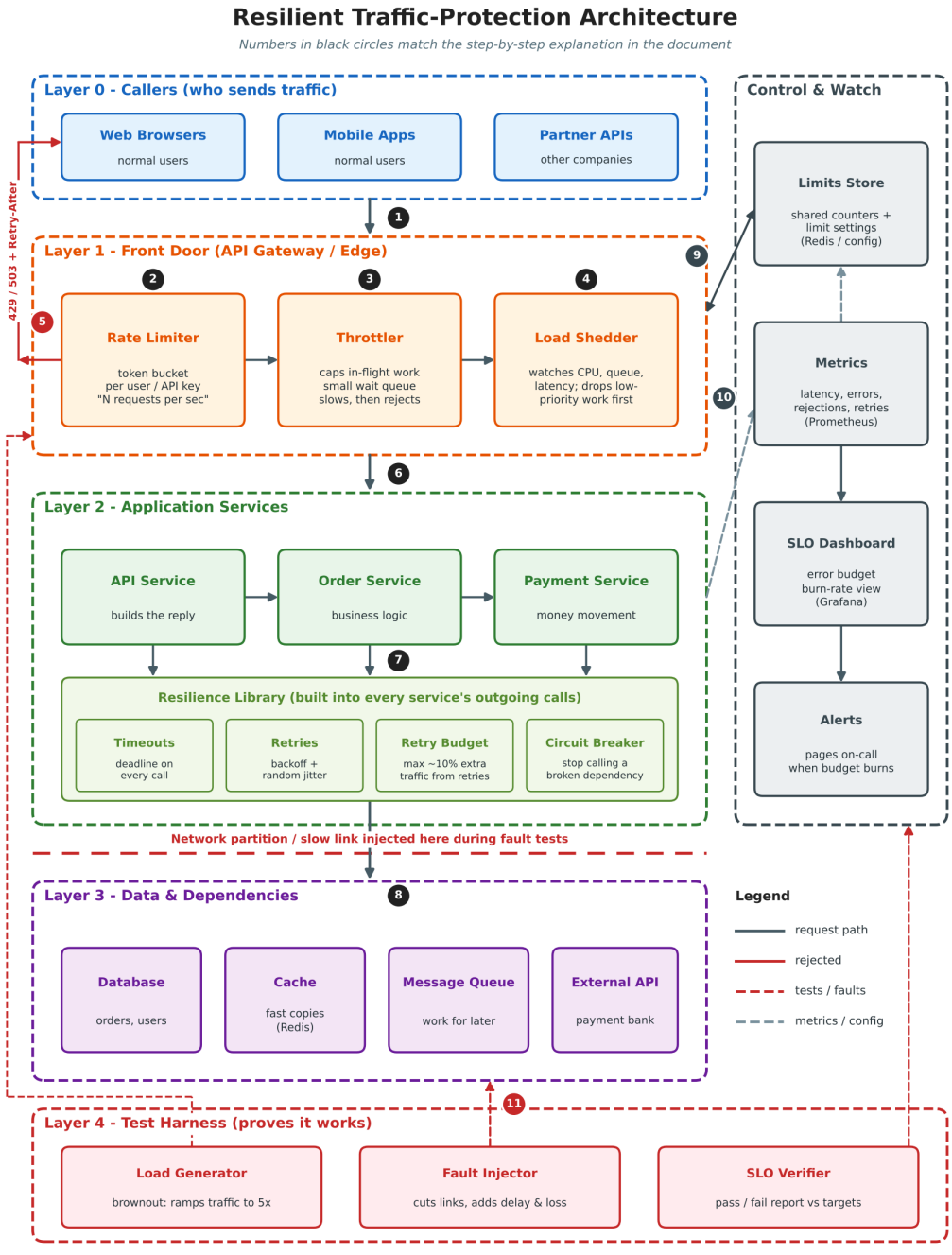


Figure 1. Resilient traffic-protection architecture (numbers match the walkthrough below)

4.1 How to Read the Diagram

Visual element	Meaning
Dashed outer box	A layer (a group of components with a similar job)
Solid rounded box	One component
Solid grey arrow	The normal path a request takes
Solid red arrow	A request being rejected and sent back to the caller
Dashed red arrow	Test traffic or faults injected by the test harness
Dashed grey arrow	Metrics being reported, or settings being read
Double-headed dark arrow	Two-way traffic: reading and updating shared counters
Red dashed horizontal line	The place where network partitions and slow links are injected during tests
Numbered circle	A step in the walkthrough in section 4.3

4.2 The Layers, One by One

Layer 0: Callers (blue)

These are everyone who sends requests: **web browsers**, **mobile apps** and **partner APIs** (other companies' systems). They are outside our control, which is exactly why the system must protect itself. Well-behaved callers follow the Retry-After hints; badly behaved ones are contained by the rate limiter.

Layer 1: Front Door / API Gateway (orange)

The single entrance for all traffic. It contains three checks in a fixed order, from cheapest to most context-aware: the **Rate Limiter**, the **Throttler**, and the **Load Shedder**. A request must pass all three to enter. The order matters: cheap checks run first, so rejecting bad or excess traffic costs almost nothing.

Layer 2: Application Services (green)

The programs that do the real work: the **API Service** builds the reply, the **Order Service** handles business logic, and the **Payment Service** moves money. They call each other from left to right. Beneath them is the **Resilience Library**, a shared piece of code that wraps every outgoing call with **Timeouts**, **Retries**, a **Retry Budget** and a **Circuit Breaker**. Because it is shared, every service gets the same protection without each team re-building it.

Layer 3: Data and Dependencies (purple)

Things the services rely on: the **Database** (permanent records such as orders and users), the **Cache** (fast copies of popular data, also used for fallbacks), the **Message Queue** (work that can be finished later), and the **External API** (an outside company, such as a payment bank). These are the most common places where slowness and partitions occur, which is why the partition line sits just above them.

Control and Watch column (dark grey)

The **Limits Store** holds shared counters and settings. **Metrics** collects numbers from every layer. The **SLO Dashboard** turns those numbers into "are we keeping our promises?" views. **Alerts** notify the on-call engineer when the error budget is burning too fast.

Layer 4: Test Harness (red)

The **Load Generator** creates surges up to 5x (brownout tests) and feeds them into the front door. The **Fault Injector** breaks the network between services and dependencies. The **SLO Verifier** reads the metrics and decides pass or fail.

4.3 Walkthrough of Every Numbered Point

#	Where	What happens, in simple terms
1	Callers to Front Door	Every request from every caller enters through one door, the API gateway. The gateway stamps it with a request ID (for tracing) and a deadline (for example, 2 seconds).
2	Rate Limiter	Checks the caller's token bucket in the shared Limits Store. Token available: continue. No token: reject right away.
3	Throttler	Checks whether there is a free work slot. If all slots are busy, the request may wait briefly in a small queue; if it cannot get a slot in time, it is rejected.
4	Load Shedder	Looks at system health (CPU, queue wait, latency) and the request's priority. Under stress, low-priority requests are dropped or served from cache so critical ones survive.
5	Rejection path (red)	Any request rejected at 2, 3 or 4 is sent back quickly with 429 or 503 and a Retry-After hint. The caller knows when to try again, and the system spends almost nothing on it.
6	Front Door to Services	Accepted requests reach the application services, carrying their deadline and priority with them.

#	Where	What happens, in simple terms
7	Services to Resilience Library	Whenever a service needs to call anything else, the call goes through the library: timeout set from remaining deadline, circuit breaker checked, retries only if safe and within budget.
8	Library to Dependencies	The protected call reaches the database, cache, queue or external API. This link is where the partition is injected during fault tests. If it fails, the library's timeout, breaker and fallback contain the damage.
9	Gateway and Limits Store	The gateway reads and updates shared counters and reads its limit settings, so all gateway servers enforce one consistent limit. If the store is unreachable, gateways use safe local limits.
10	Services to Metrics	Every service reports latency, errors, rejections, retries, breaker states and fallback use. Metrics feed the SLO Dashboard, which feeds Alerts.
11	Test Harness	The Load Generator pushes up to 5x traffic into the front door; the Fault Injector cuts and slows the dependency links; the SLO Verifier reads the metrics and produces the pass or fail report.

4.4 Why the Architecture Is Shaped This Way

- **Protection at the edge first.** Rejecting excess traffic at the front door is cheap. Rejecting it deep inside, after database calls have been made, is expensive and wasteful.
- **Protection again inside.** Services also limit and shed for themselves, so a surge of internal calls (for example, from a retry bug) cannot overwhelm them either. This is called defense in depth.
- **One shared resilience library.** Consistent timeouts, retries and breakers everywhere, configured in one place, instead of each team inventing their own.
- **Shared state for limits.** One Limits Store keeps limits fair across many gateway servers.
- **Fallback data sources nearby.** The cache and message queue let services give useful answers even when the database or external API is unreachable.
- **Everything is measured.** Every decision is counted, so the team can prove the SLOs held and tune the thresholds.

5. Workflow Explained End to End

This section follows the work in three ways: first the life of one request, then what happens over time during a 5x surge, and finally what happens during a network partition.

5.1 The Life of One Request

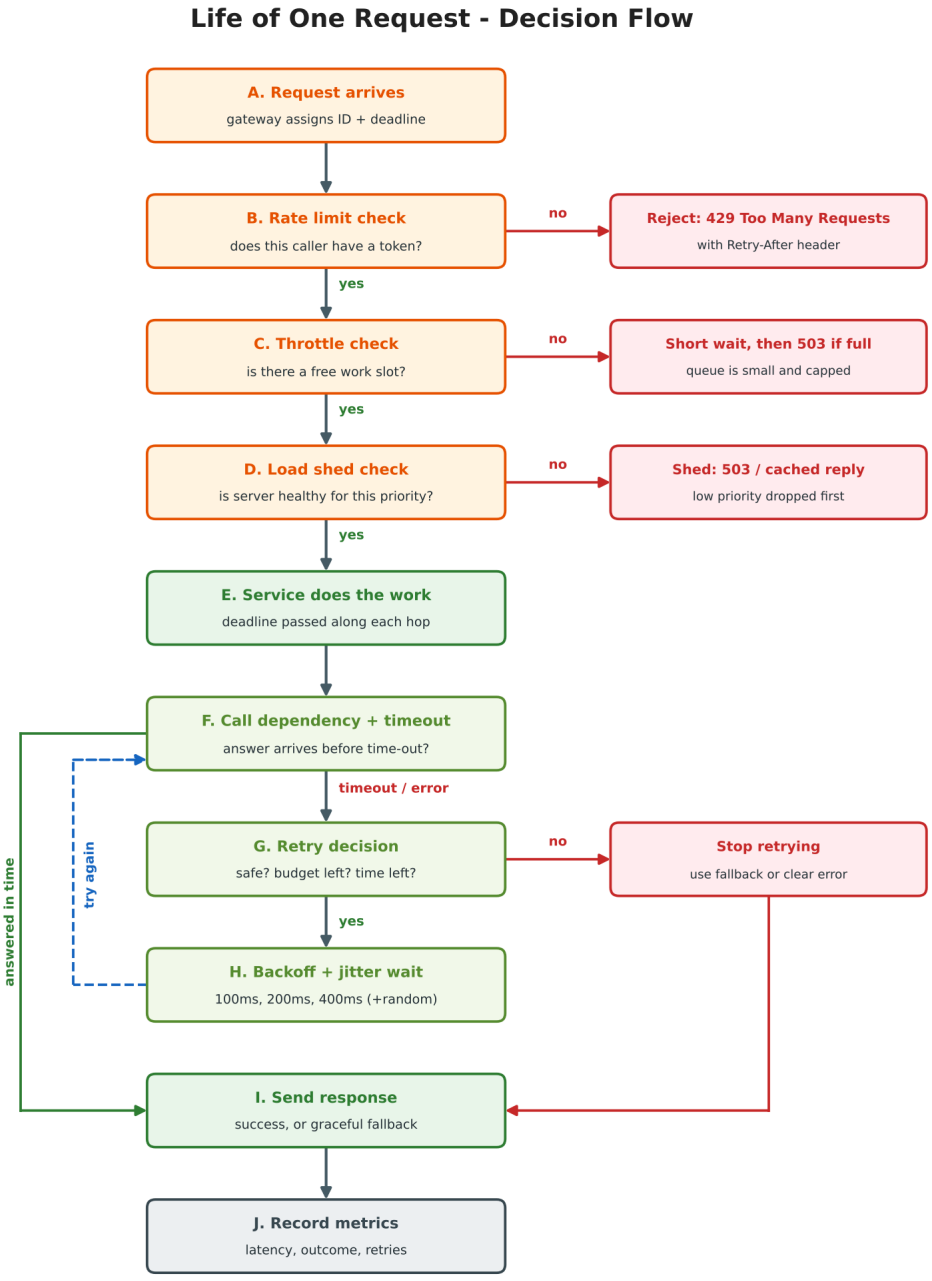


Figure 2. Decision flow for a single request (letters match the steps below)

Step	What happens, in detail
A. Request arrives	The caller sends a request. The gateway gives it a unique ID (so it can be traced through every service), a deadline (how long it may take in total), and a priority (based on what kind of request it is, for example checkout = Critical).
B. Rate limit check	The gateway asks: "Does this caller have a token left?" If yes, one token is used and the request moves on. If no, the request is answered instantly with 429 Too Many Requests and a Retry-After value. Cost to the system: nearly zero.
C. Throttle check	The gateway asks: "Is there a free work slot?" If yes, the request takes a slot. If not, it may wait a few milliseconds in a small queue. If the queue is full or the wait runs out, it gets a 503 with Retry-After.
D. Load shed check	The gateway asks: "Given current system health, is this request important enough?" At Green everything passes. As pressure rises, low then normal priority requests are dropped or answered from cache. Requests whose deadline is nearly over are also dropped here, since finishing them would be wasted effort.
E. Service does the work	The request reaches the services. Each service passes along the remaining deadline, so later steps know exactly how much time is left. Services also have their own internal throttle and shedder as a second line of defense.
F. Call a dependency with a timeout	When the service needs the database, cache, another service or an external API, the resilience library first checks the circuit breaker (if it is open, skip straight to the fallback). Otherwise it makes the call with a timeout no longer than the remaining deadline. If the answer comes back in time, go to step I.
G. Retry decision	If the call timed out or failed, the library asks four questions: Is the error temporary? Is the operation safe to repeat (or protected by an idempotency key)? Is there retry budget left? Is there enough time left before the deadline? If any answer is no, stop and use the fallback.
H. Backoff and jitter wait	If a retry is allowed, wait first: about 100 ms before the 2nd try, 200 ms before the 3rd, each picked randomly between zero and that value. If the server gave a Retry-After, use that instead. Then go back to step F. At most 3 attempts in total.

Step	What happens, in detail
I. Send the response	The user receives either the real answer, a graceful fallback (cached or partial data), or a fast, clear error. In all cases the reply comes within the deadline.
J. Record metrics	The outcome is counted: success or failure, total time, how many retries, whether it was shed or limited, and whether a fallback was used. These numbers feed the SLO dashboard.

5.2 Scenario: A 5x Traffic Surge

Here is what happens over time when traffic suddenly jumps to five times normal. Assume normal traffic is 1,000 requests per second and the system can comfortably handle about 2,500.

Time	Traffic	What the system does	What users experience
T+0 s	1,000/s (1x)	Everything Green. All requests accepted.	Normal, fast responses.
T+10 s	Jumps to 5,000/s	Rate limiter immediately caps any single caller who is far above quota (for example, a buggy partner script). Their extra traffic gets 429s.	The noisy caller is slowed down; everyone else is unaffected.
T+12 s	About 4,000/s still arriving	Throttler slots fill up. A small queue forms. Requests that cannot get a slot within the short wait get 503.	Most users still fast; a few see "try again".
T+15 s	Sustained	Load shedder detects rising queue wait and CPU: moves to Yellow, then Orange. Low-priority work (analytics, prefetch) is dropped; normal-priority work like recommendations is served from cache.	Checkout and login stay fast. Some optional sections show slightly older data.
T+15 s onward	Sustained	Clients that got 429/503 follow Retry-After with jittered backoff, so their retries arrive spread out instead of in a wave. The retry budget stops services from multiplying load internally.	Retried requests mostly succeed on the next try.

Time	Traffic	What the system does	What users experience
T+120 s	Surge fades	Signals improve. The shedder steps back down level by level after each cool-down period, so it does not flap.	Everything back to normal.
Throughout		The system works at its full safe capacity (around 2,500/s) the whole time. It never tries to do more and collapse.	Critical SLOs (success rate and latency) stay within target.

What would have happened without protection

All 5,000 requests per second would be accepted. Queues would grow without limit, every request would slow down, timeouts would start firing, clients would retry instantly (pushing traffic toward 10,000 per second), memory would run out and servers would crash. As servers crashed, the remaining ones would get even more traffic and crash too. That chain reaction is a cascading failure, and it would have taken the whole service down, including checkout.

5.3 Scenario: A Network Partition

Here the traffic is normal, but the link between the Payment Service and the external payment API becomes very slow, then completely cut, for 60 seconds.

Phase	What the system does	Why it matters
1. Link slows down	Calls to the payment API start taking longer. Per-call timeouts (for example, 800 ms) cut off slow calls instead of waiting forever.	Payment Service threads are freed quickly, so it does not freeze.
2. Some calls fail	The library retries timed-out calls that are safe to retry (payments carry an idempotency key so they are never charged twice), with backoff and jitter.	Brief blips are smoothed over invisibly.
3. Failures pile up	The retry budget is used up, so further retries stop. The circuit breaker sees more than 50% failures and opens.	No retry storm; no extra pressure on the broken link.

Phase	What the system does	Why it matters
4. Breaker open	Calls to the payment API fail instantly. The fallback places the payment in the message queue as "pending" and tells the user the order is received and payment is being confirmed. Other features (browsing, cart, search) are untouched.	Users get a quick, honest answer. The failure does not spread to other services.
5. Partition heals	After the cool-down, the breaker goes half-open and sends a few trial calls. They succeed, so it closes.	Normal service resumes automatically, without anyone needing to restart anything.
6. Catch up	Queued payments are processed at a controlled rate (itself rate limited) so the recovering payment API is not flooded.	Recovery does not cause a second overload.

The same pattern applies if the database, cache or another internal service is partitioned: timeouts notice the problem, retries handle small blips, the budget and breaker stop the pile-on, and fallbacks keep users served.

5.4 The Engineering Workflow: How the Project Was Delivered

Beyond the runtime flow, the project itself followed a clear sequence of work:

1. **Define the SLOs.** Agree on targets with product owners, for example 99.9% success and p99 latency under 500 ms for critical endpoints.
2. **Measure capacity.** Load-test each service to find where latency starts to climb. This number sets the throttle limits and shedding thresholds.
3. **Classify traffic.** Tag every endpoint with a priority (Critical, High, Normal, Low) and mark which operations are safe to retry.
4. **Add timeouts everywhere.** Audit every outgoing call (HTTP, database, cache) and set connect and request timeouts; add deadline propagation.
5. **Build the resilience library.** Retries with exponential backoff and full jitter, retry budget, circuit breakers and fallback hooks, in one shared package.
6. **Add edge protection.** Rate limiter with shared token buckets, concurrency throttle and priority-aware load shedder in the gateway; repeat the throttle and shedder inside services.
7. **Instrument everything.** Metrics for every decision; SLO dashboard with error budget and burn-rate alerts.

8. **Run brownout tests.** Ramp to 2x, 3x and 5x; tune thresholds until SLOs hold.
9. **Run fault-injection tests.** Inject latency, packet loss and full partitions on each dependency link; fix any cascade found.
10. **Automate and repeat.** Add the tests to the release process so the protections stay working as the code changes.

6. Testing and Proof

The project statement says the results were "measured by brownout and fault-injection tests". This section explains what those tests are, what was measured, and how pass or fail was decided.

6.1 Brownout Tests (Overload)

A brownout test turns the pressure up gradually and watches whether the system dims gracefully, like lights in a brownout, rather than going completely dark.

Stage	Load	Duration	What is checked
Baseline	1x normal	10 min	Normal SLIs recorded for comparison
Ramp 1	2x	10 min	No shedding of critical traffic; latency within target
Ramp 2	3x	10 min	Low-priority shedding begins; critical SLOs still met
Peak	5x	15 min	Critical SLOs met; no crashes; memory and CPU stable
Capacity squeeze	5x with some servers removed	10 min	Shedding increases, but still no collapse
Recovery	Back to 1x	10 min	Return to baseline quickly, without flapping

6.2 Fault-Injection Tests (Network Partitions)

Fault injected	Where	Expected behaviour
Added latency (e.g. +500 ms, +2 s)	Service to database, service to external API	Timeouts fire; fallbacks used; no thread pile-up
Packet loss (e.g. 10%, 30%)	Between services	Retries with jitter recover most calls; retry budget not exceeded
Full partition (link cut)	Each dependency in turn	Circuit breaker opens; fast fallback; other features unaffected

Fault injected	Where	Expected behaviour
Dependency crash and restart	Database replica, cache node	Brief errors absorbed; automatic recovery when it returns
Limits store unreachable	Gateway to Limits Store	Gateways switch to safe local limits; no outage
Combined: 5x surge plus partition	Whole system	Critical SLOs held; no cascade

6.3 Pass or Fail Criteria

The SLO Verifier checks these conditions after every test. The exact numbers depend on each team's own SLOs; the values below show the kind of targets used.

Criterion	Example target	How it is measured
Critical success rate	At least 99.9% during the test	Successful critical responses / all critical requests
Critical latency	p99 under 500 ms	Latency histogram from metrics
No cascading failure	Zero services outside the faulted area breach their SLO	Per-service SLIs compared with baseline
No crashes	Zero out-of-memory or restart events	Container and process restart counts
Retry amplification	Retries under 10% of total traffic	Retry counter / request counter
Fast rejection	Rejected requests answered in under 10 ms	Latency of 429 and 503 responses
Recovery time	Back to baseline within a few minutes of the fault ending	Time until SLIs match baseline

> **What "zero cascading failures" means in measurable terms**
>
> When one dependency was broken or one part of the system was overloaded, every other service kept meeting its own SLO. The damage stayed in the place where the fault was injected, and nothing failed in a chain reaction.

7. Quick Reference

Technique	Simple question it answers	Typical setting	Response when triggered
Rate limiting	Is this caller over their fair share?	Token bucket, e.g. 100/s, burst 200	429 + Retry-After
Throttling	Are we already at full safe workload?	Max in-flight from load tests; tiny queue	Short wait, then 503
Load shedding	Are we stressed, and is this important?	Priority classes + health thresholds	503 or cached answer
Timeouts	Has this wait gone on too long?	Per dependency; within remaining deadline	Cancel, free resources
Retries + backoff	Was it a temporary glitch?	3 attempts, 100 ms base, full jitter	Wait and retry
Retry budget	Are retries adding too much load?	10% of requests	Stop retrying
Circuit breaker	Is this dependency down?	Open at 50% errors, 10 s cool-down	Fail fast
Fallback	What is the best answer we can still give?	Cache, default, queue for later	Partial or older data

Key Takeaways

- Say "no" quickly to some work so that you can say "yes" reliably to the rest.
- Never wait forever: every call has a timeout, and deadlines travel with the request.
- Retry gently: only temporary errors, only safe operations, with growing random waits, within a budget, at one layer.
- Protect what matters most: priorities decide what to drop first.
- Contain failures: circuit breakers and fallbacks stop one broken part from breaking the rest.
- Prove it: brownout and fault-injection tests turn "we think it is resilient" into measured evidence.